

ARX Development Environment Setup Guide

Document Version: V25.10.01

Status: Released

Last Updated: 30 October 2025

Prepared By: ARX RTOS Team

1.0 Introduction

1.1 Purpose

This guide provides instructions for setting up the ARX RTOS development environment across supported development platforms.

It covers the installation and configuration of the required development tools, toolchains, IDE components, and the basic workflow required to build and run ARX RTOS on supported boards or simulators.

1.2 Audience

This guide is intended for:

- ARX RTOS developers
- Students working with ARX RTOS
- Integrators working with supported ARX RTOS platforms

1.3 Scope

This guide covers the following areas:

- Development environment prerequisites
- Required development tools and toolchains
- Windows development environment setup
- Linux development environment setup

- Eclipse IDE configuration
- ARX RTOS project configuration
- Build and flashing procedures

1.4 Limitations

The procedures in this guide are intended for the development environments and platforms supported by the corresponding ARX RTOS release.

Tool versions, IDE behavior, compiler configuration, and target-specific procedures may vary between ARX RTOS releases and supported platforms.

2.0 Prerequisites

Before setting up the ARX RTOS development environment, ensure that the development host meets the following requirements.

2.1 Supported Operating Systems

- Windows 10 / 11
- Linux (Ubuntu 20.04 LTS / 22.04 LTS)

2.2 Required Development Tools

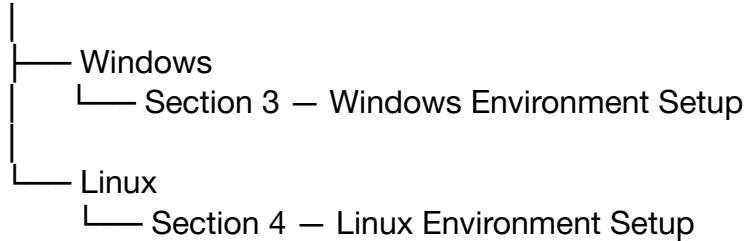
The following tools are required for ARX RTOS development:

Tool / Component	Purpose
GNU Make	Build system
ARM GCC Toolchain	Building ARX RTOS for ARM Cortex-A/R/M targets
RISC-V GCC Toolchain	Building ARX RTOS for RISC-V RV32 / RV64 targets
QEMU	Target simulation
Eclipse IDE	Integrated development environment

2.3 Development Environment Selection

Select the setup procedure according to the operating system used on the development host:

Development Host



3.0 Environment Setup(Windows)

This section describes the procedure for preparing a Windows development host for ARX RTOS development.

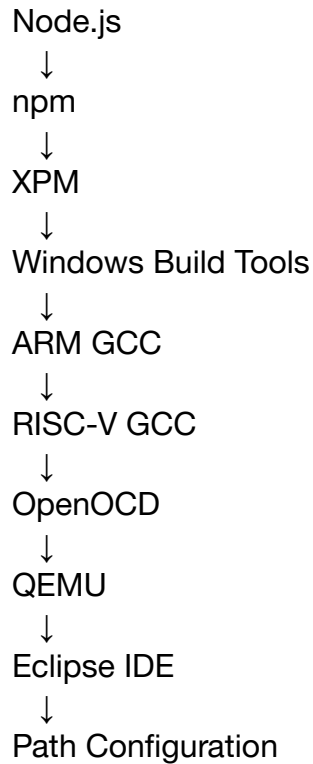
The setup includes the installation of the required package-management tools, build tools, cross-compilers, debugging tools, QEMU, and Eclipse IDE.

3.1 Windows Development Environment

Component	Purpose
Node.js	Provides the Node.js runtime and npm package manager
npm	Package management
XPM	Installation and management of xPack development tools
Windows Build Tools	Required Windows build utilities
ARM GCC	ARM Cortex-M cross-compilation
RISC-V GCC	RISC-V cross-compilation
OpenOCD	Target programming and debugging
QEMU	Target simulation
Eclipse IDE	Integrated development environment

3.2 Installation Sequence

Install the components in the following order:



3.3 Installation Requirements

1. Log in to Windows using an account with sufficient privileges to install development software.
2. Open PowerShell when instructed by the installation procedure.
3. Ensure that an active Internet connection is available for downloading the required tools and packages.
4. Use the tool versions specified or validated for the applicable ARX RTOS release.

Note: Tool versions may change between ARX RTOS releases. The versions specified in this guide should be treated as the versions applicable to the corresponding ARX RTOS release.

3.4 Install Node.js

1. Download the Installer:
Download the Node.js installer from the official Node.js download page.
<https://nodejs.org/en/download>
2. Run the Setup:
Launch the installer and follow the prompt instructions. Ensure you check the option to install all necessary tools for Native Modules (this will automatically install Chocolatey and additional required build tools for Windows).

3. Verify Installation:
Open a new PowerShell or Command Prompt window and execute:

```
node -v
```

```
npm -v
```

Verification: Ensure both commands return their respective version numbers without errors.

4. Troubleshooting — Execution Policy Restriction:

If Windows blocks tool execution or installation scripts during this process, update your script execution permissions by running the following command in PowerShell:

```
Set-ExecutionPolicy -ExecutionPolicy RemoteSigned -Scope CurrentUser
```

3.5 Install XPM

3.5.1 Install XPM

Execute the following command in PowerShell

```
PS C:\Windows\system32> npm install --global xpm@latest
```

3.5.2 Verify the Installation

```
PS C:\Windows\system32> npx xpm --version
```

3.6 Install Windows Build Tools

Windows Build Tools provide the build utilities required by the ARX RTOS development environment.

3.6.1 Install Windows Build Tools

Execute the following command in PowerShell

```
PS C:\Windows\system32> npx xpm install --global @xpack-dev-tools/windows-build-tools@latest
```

3.6.2 Verify the Installation

The current source document does not specify a verification command or expected output for the Windows Build Tools. Please verify the respective installation directory.

3.7 Install ARM GCC Toolchain

3.7.1 Installation

Execute the following command in PowerShell

```
PS C:\Windows\system32> npx xpm install --global @xpack-dev-tools/arm-none-eabi-gcc@latest
```

3.7.2 Verify the Installation

The current source document does not specify a verification command or expected output for the Windows Build Tools. Please verify the respective installation directory.

3.7.3 Toolchain Path

The ARM GCC toolchain path will be required later when configuring the Eclipse project.

For example enquire the location as-

```
C:/Users/<user>/AppData/Roaming/xPacks/@xpack-dev-tools/arm-none-eabi-gcc/<version>/content/bin
```

3.8 Install RISC-V GCC Toolchain

The RISC-V GCC toolchain is required to build ARX RTOS applications for RISC-V RV32 / RV64 targets.

3.8.1 Installation

Execute the following command in PowerShell

```
PS C:\Windows\system32> npx xpm install --global @xpack-dev-tools/riscv-none-elf-gcc@latest
```

3.8.2 Verify the Installation

The current source document does not specify a verification command or expected output for the Windows Build Tools. Please verify the respective installation directory.

3.8.3 Toolchain Path

The RISC-V GCC toolchain path will be required later when configuring the Eclipse project.

For example enquire the location as-

```
C:/Users/<user>/AppData/Roaming/xPacks/@xpack-dev-tools/riscv-none-elf-gcc/<version>/content/bin
```

3.9 Install OpenOCD

OpenOCD is used for programming and debugging supported target hardware.

3.9.1 Installation

Execute the following command in PowerShell

```
PS C:\Windows\system32> npx xpm install --global @xpack-dev-tools/openocd@latest
```

3.9.2 Verify the Installation

The current source document does not specify a verification command or expected output for the Windows Build Tools. Please verify the respective installation directory.

3.10 Install QEMU

QEMU is used to simulate supported target platforms and allows ARX RTOS applications to be built and tested without physical target hardware.

3.10.1 Installation

Download the Windows installer from the official QEMU website.

<https://www.qemu.org/download/>

1. Open the QEMU download page.
2. Select Windows.
3. Select the appropriate 64-bit installer.
4. Download the QEMU Windows installer.
5. Run the installer and complete the installation.

3.11 Install Eclipse IDE

Eclipse IDE provides the graphical development environment used to create, configure, build, and manage ARX RTOS projects.

3.11.1 Download Eclipse IDE

Download Eclipse from the official Eclipse website.

https://www.eclipse.org/downloads/?utm_source=chatgpt.com

3.11.2 Install Eclipse

1. Launch the Eclipse installer.
2. Select Eclipse IDE for Embedded C/C++ Developers.
3. Follow the installation wizard.
4. Complete the installation using the required installation location.

3.11.3 Eclipse Toolchain Configuration

After Eclipse is installed, the compiler and build-tool paths must be configured for the ARX RTOS project.

The specific configuration of the ARM GCC toolchain path is covered later in the Project Configuration section.

3.12 Configure Windows Environment Paths

After installing the required development tools, configure the tool paths used by the ARX RTOS project in Eclipse.

The compiler, Windows build tools, and OpenOCD as the paths that must be configured in the Eclipse project.

3.12.1 Required Tool Paths

Tool	Purpose
ARM GCC	ARM target compilation
RISC-V GCC	RISC-V target compilation
Windows Build Tools	Build utilities
OpenOCD	Target programming and debugging

4.0 Environment Setup(Linux)

This section describes the procedure for preparing a Linux development host for ARX RTOS development.

The Linux setup in the existing guide includes the installation of the required system packages, ARM GCC toolchain, Eclipse IDE, and the associated project configuration.

4.1 Linux Development Environment

The following components are required:

Component	Purpose
System packages	Provide the basic Linux development environment
Java Runtime Environment	Required by Eclipse
ARM GCC Toolchain	ARM Cortex-M cross-compilation
Eclipse IDE	Integrated development environment

4.2 Installation Sequence

Follow the installation steps in the order below:

System Update



Java Runtime Environment



ARM GCC Toolchain



Eclipse IDE



Eclipse Toolchain Configuration

4.3 Open a Terminal

Open a Linux terminal before starting the installation procedure.

4.4 Update the Linux Development Environment

Before installing the ARX RTOS development tools, update the package information on the Linux development host.

Open a terminal and run:

```
ARXMACH@stl:~$ sudo apt-get update -y
```

4.5 Install Java Runtime Environment

Java is required for the Eclipse-based development environment used by ARX RTOS.

4.5.1 Install Java

Open a terminal and run:

```
ARXMACH@stl:~$ sudo apt install default-jre
```

4.5.2 Verify the Installation

```
ARXMACH@stl:~$ java --version
```

4.6 Install ARM GCC Toolchain

The ARM GCC toolchain is required to build ARX RTOS applications for A/R/M targets.

4.6.1 Install ARM GCC

Open a terminal and run:

```
ARXMACH@stl:~$ sudo apt-get install -y gcc-arm-none-eabi
```

4.6.2 Verify the Toolchain Path

```
ARXMACH@stl:~$ which gcc
```

4.7 Install RISC-V GCC Toolchain

The RISC-V GCC toolchain is required to build ARX RTOS applications for RISC-V RV32 / RV64 targets.

4.7.1 Install RISC-V GCC

Open a terminal and run:

```
ARXMACH@stl:~$ sudo apt-get install -y gcc-riscv64-unknown-elf
```

4.7.2 Verify the Toolchain

```
ARXMACH@stl:~$ riscv64-unknown-elf-gcc --version
```

4.8 Install Eclipse IDE

Eclipse IDE provides the graphical development environment used to create, configure, build, and manage ARX RTOS projects.

4.8.1 Download Eclipse IDE

Download the Eclipse installer from the official Eclipse website.

https://www.eclipse.org/downloads/?utm_source=chatgpt.com

4.8.2 Install Eclipse

If the Eclipse installer archive has been downloaded, extract it to the desired installation location.

```
ARXMACH@stl:~$ sudo tar -xf eclipse-inst-linux64.tar.gz -C /opt
ARXMACH@stl:~$ cd /opt/eclipse-installer
ARXMACH@stl:~$ ./eclipse-inst
```

During installation:

1. Launch the Eclipse installer.
2. Follow the installation instructions.
3. Accept the license agreement.
4. Select Eclipse IDE for C/C++ Developers.

5.0 Create and Configure an ARX RTOS Project

Editor: Eclipse IDE

Project Type: Without makefile

STEP-1

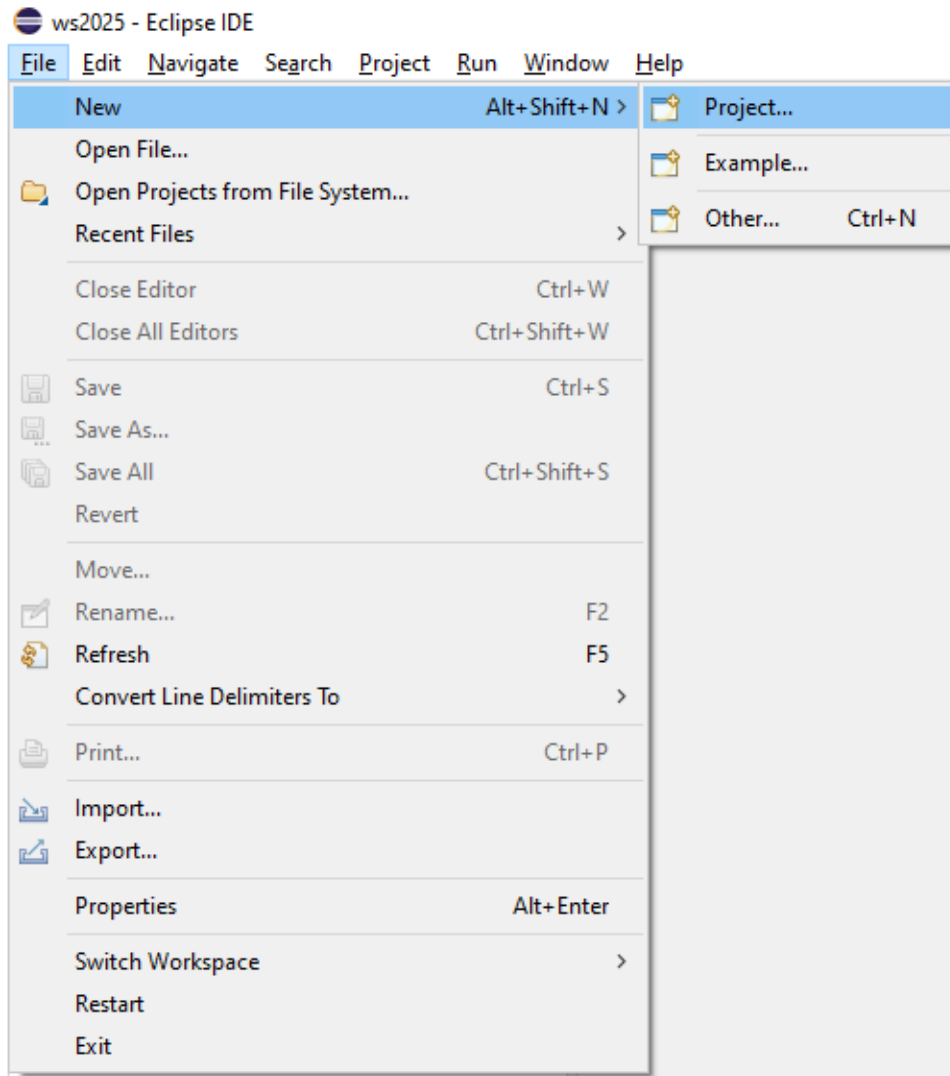


Fig 1:
File→New→Project

STEP-2

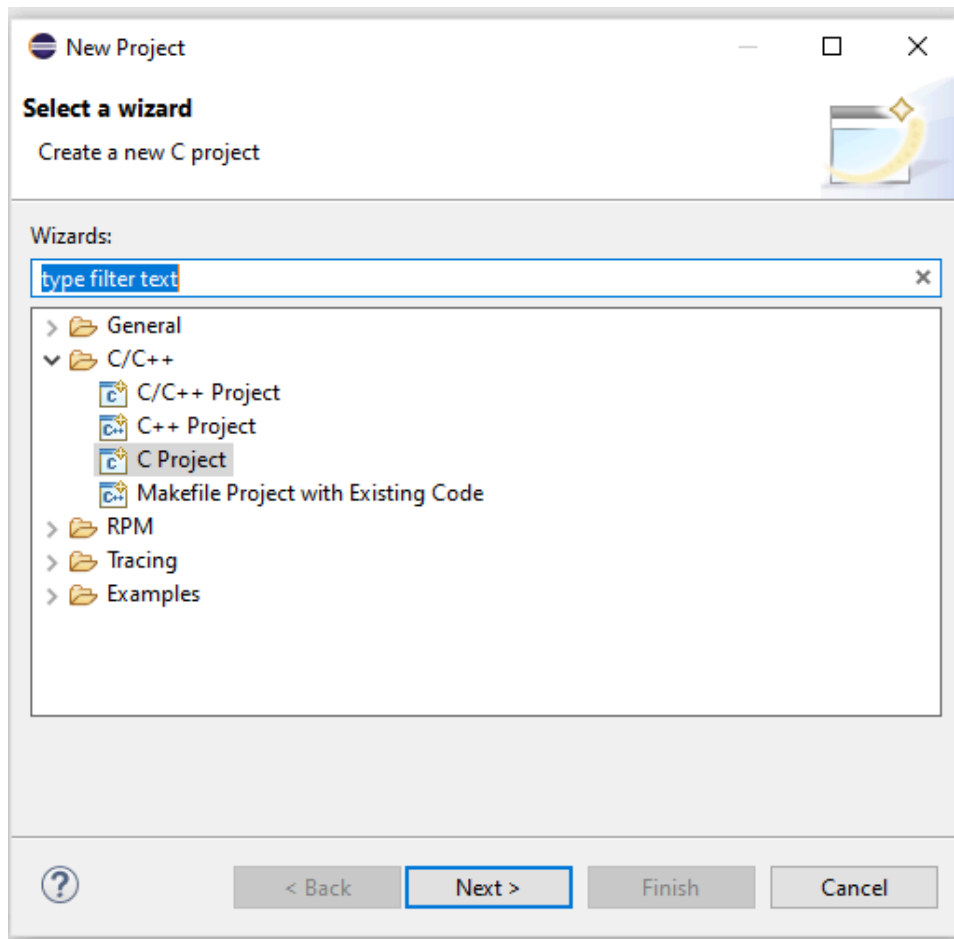


Fig 2:
File→New→Project→C Project (Select as per programming language).

STEP-3

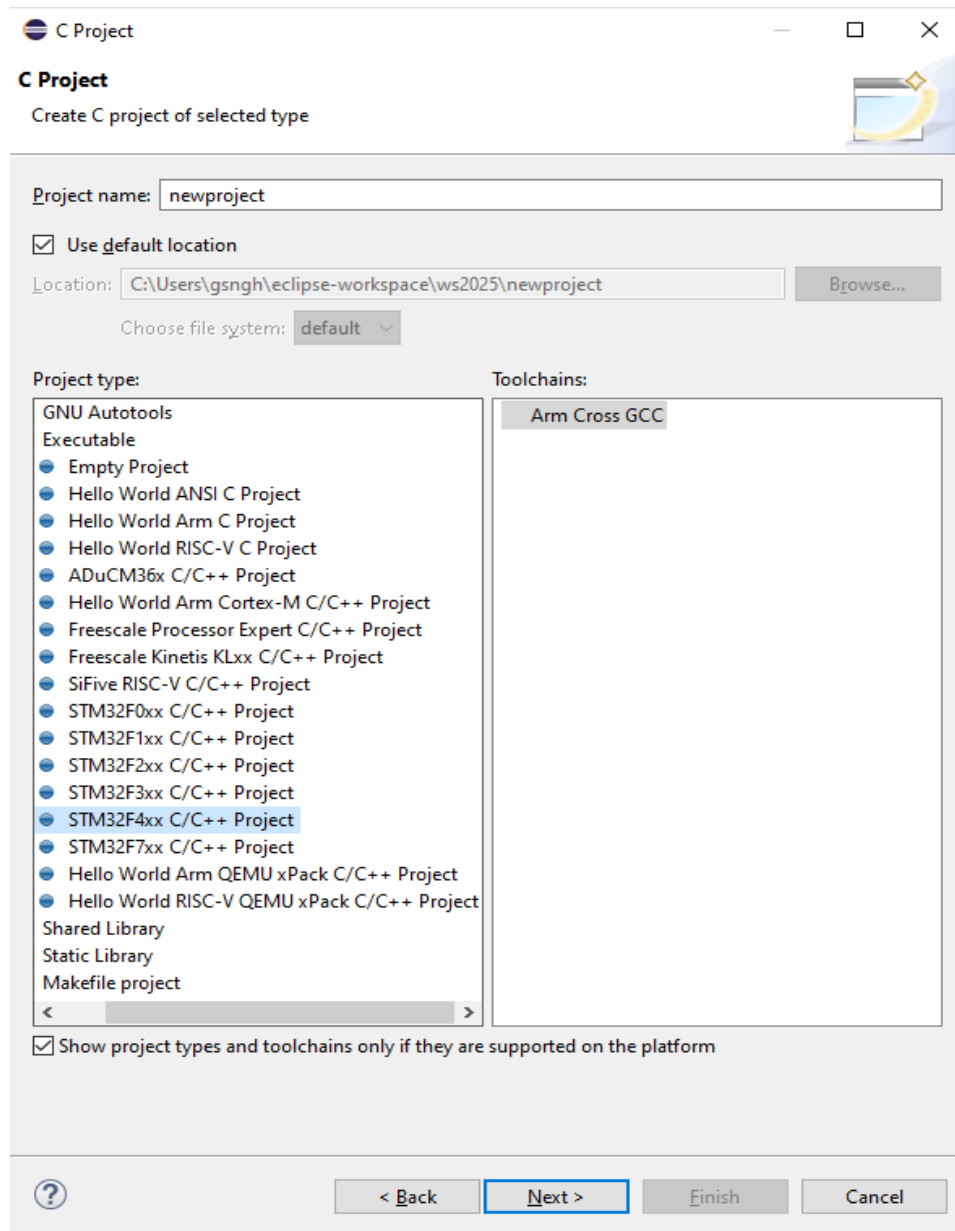


Fig 3: C Project

- I. Name the project,
- II. Select Toolchains,
- III. Select the Project type(Example- STM32F4xx C/C++ Project)

STEP-4

 C Project — □ ×

Target processor settings

Select the target processor family and define flash and RAM sizes.

Chip family:

Flash size (kB):

External clock (Hz):

Content:

Use system calls:

Trace output:

Check some warnings ☒

Check most warnings ☒

Enable -Werror ☐

Use -Og on debug ☒

Use newlib nano ☒

Exclude unused ☐

Use link optimizations ☐



Fig 4: C Project (Target Processor Setting)
Select Blinky (blink a led)

STEP-5

C Project

Folders _settings

Define the project folders and other options.

Include folder:

Source folder:

System folder:

CMSIS library folder:

C library folder:

Linker scripts folder:

< Back **Next >** **Finish** **Cancel**

Fig 5: C Project (Project folders and options).

STEP-6

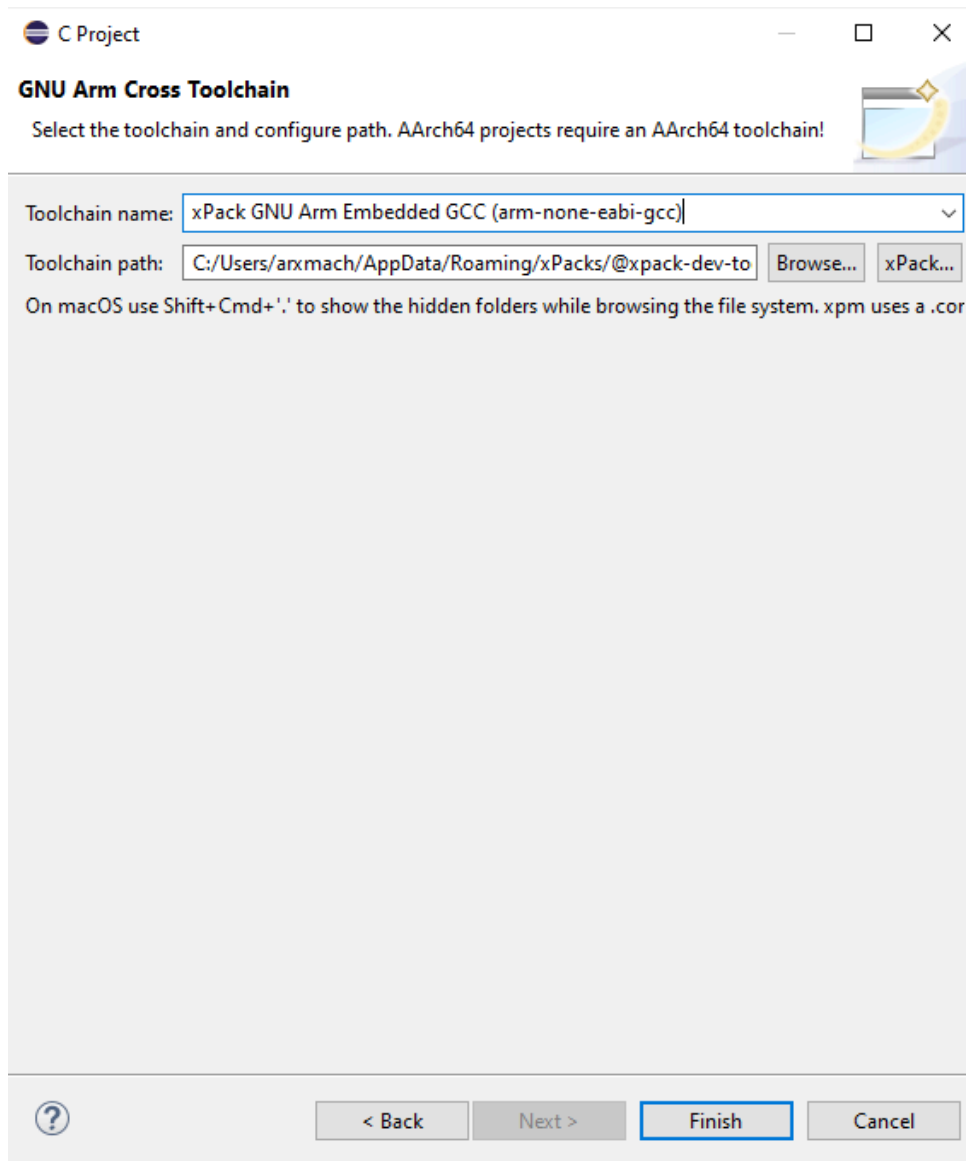


Fig 6: Setup the tool chain

- (1) Toolchain name(GCC cross compiler)
- (2) Toolchain path(GNU ARM Cross compiler path).

(Example:

C:/Users/arxmach/AppData/Roaming/xPacks/@xpack-dev-tools/arm-none-eabi-gcc/11.2.1-1.2.2/.content/bin)

Editor: Eclipse IDE
Project Type: **With Makefile**
STEP-1

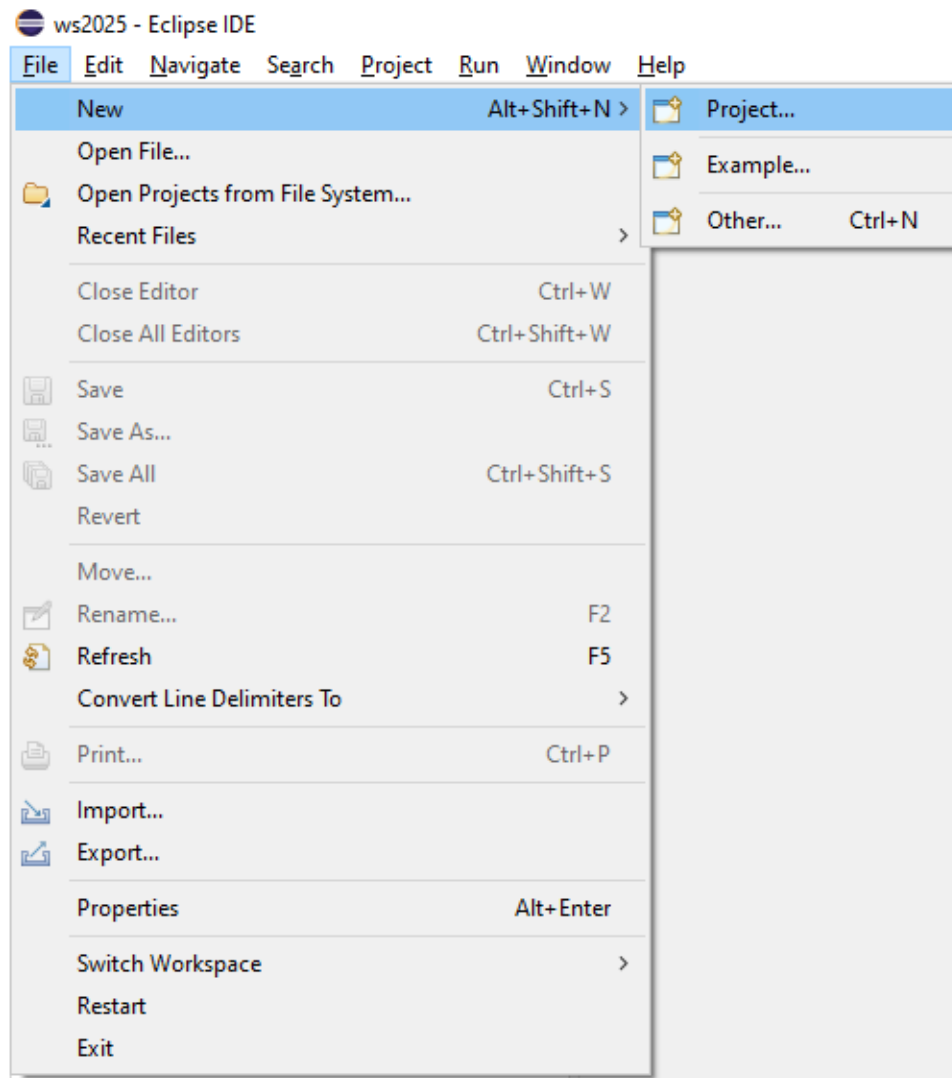


Fig 1:
File→New→Project

STEP-2

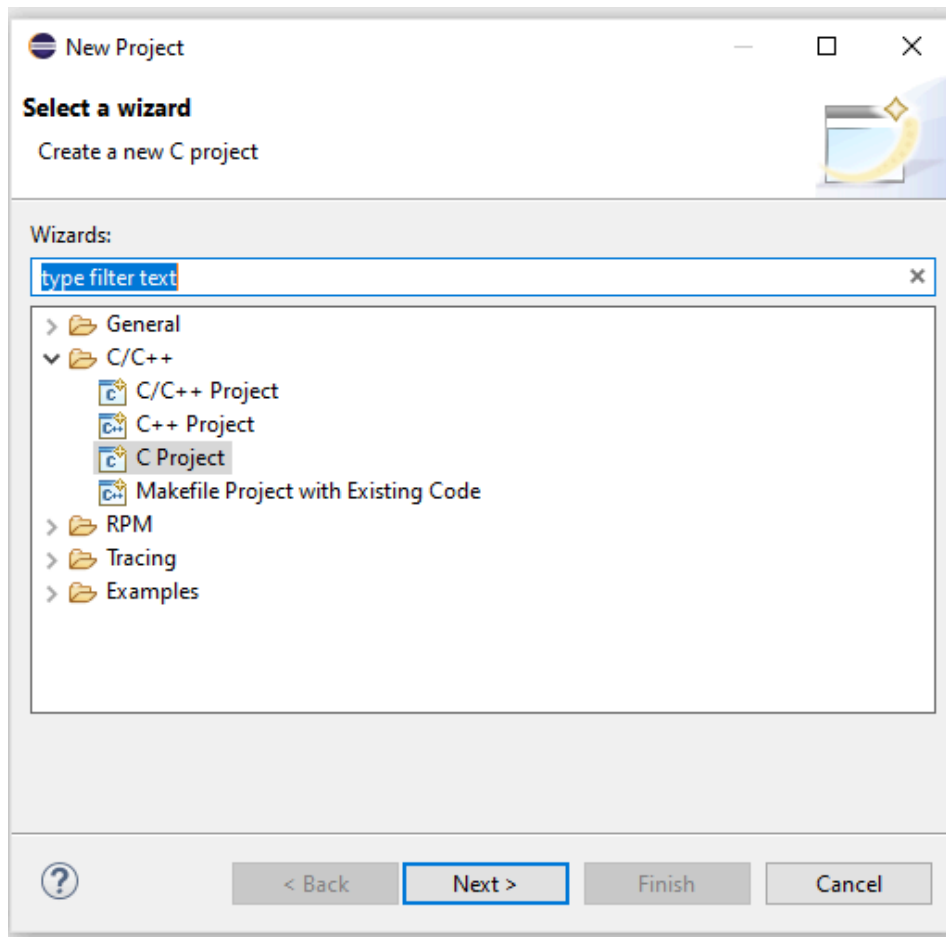


Fig 2:
File→New→Project→C Project(Select as per programming language).

STEP-3

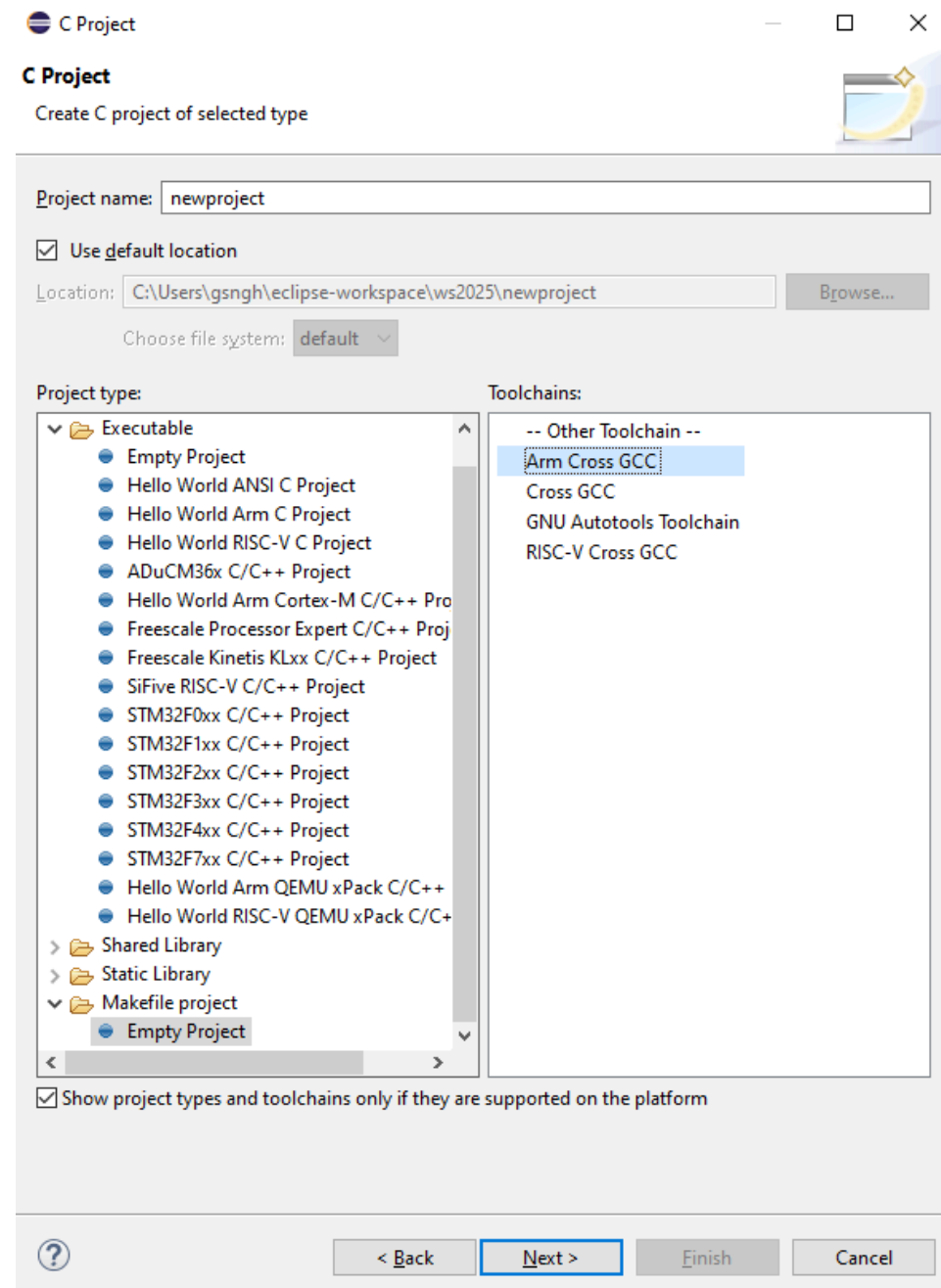


Fig 3: File→New→Project→C Project

- I. Name the project
- II. Select Makefile project (Empty Project)
- III. Select Arm Cross GCC.

STEP-4

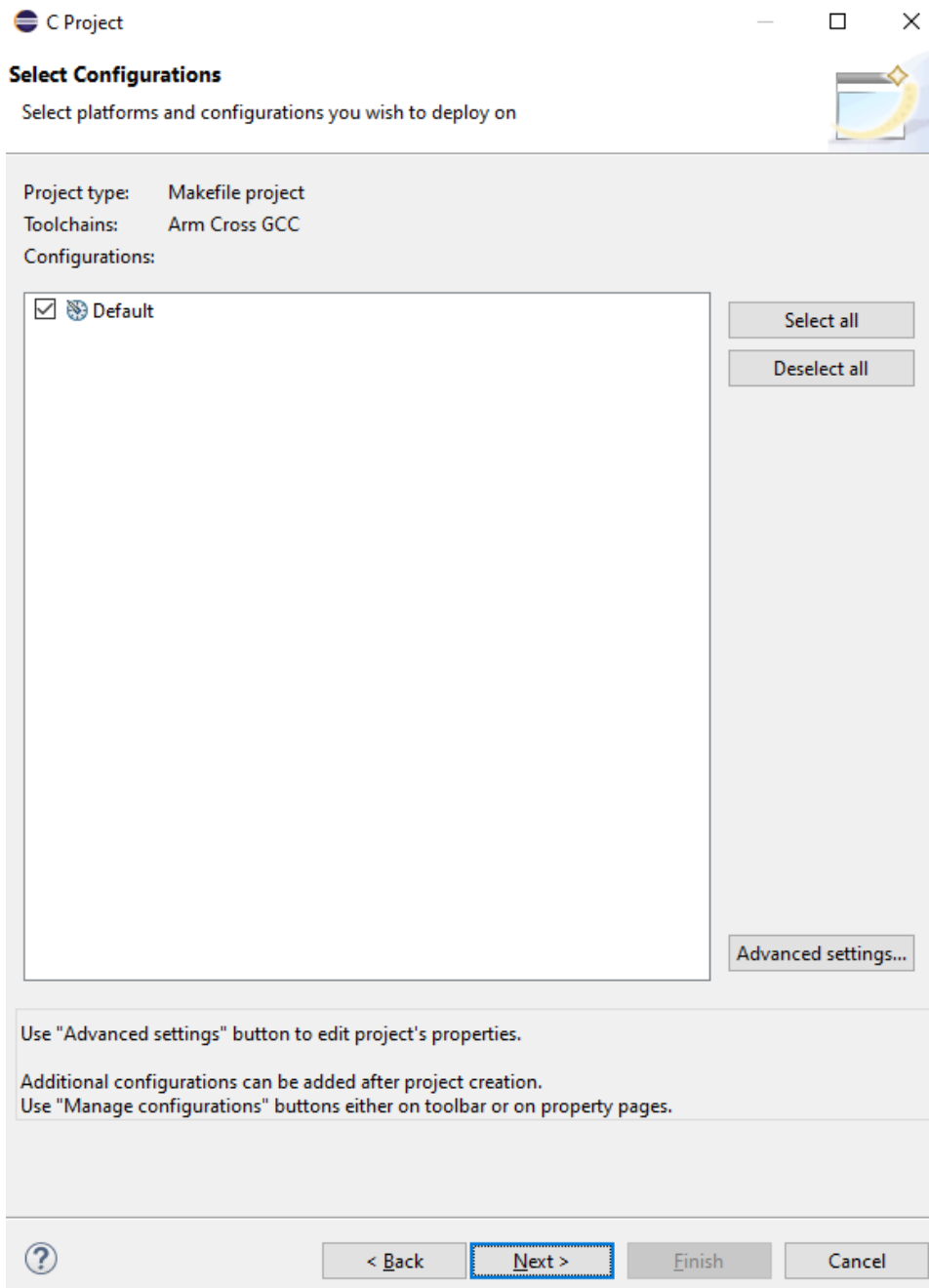


Fig 4:
File→New→Project→C Project(Follow the Next)

STEP-5

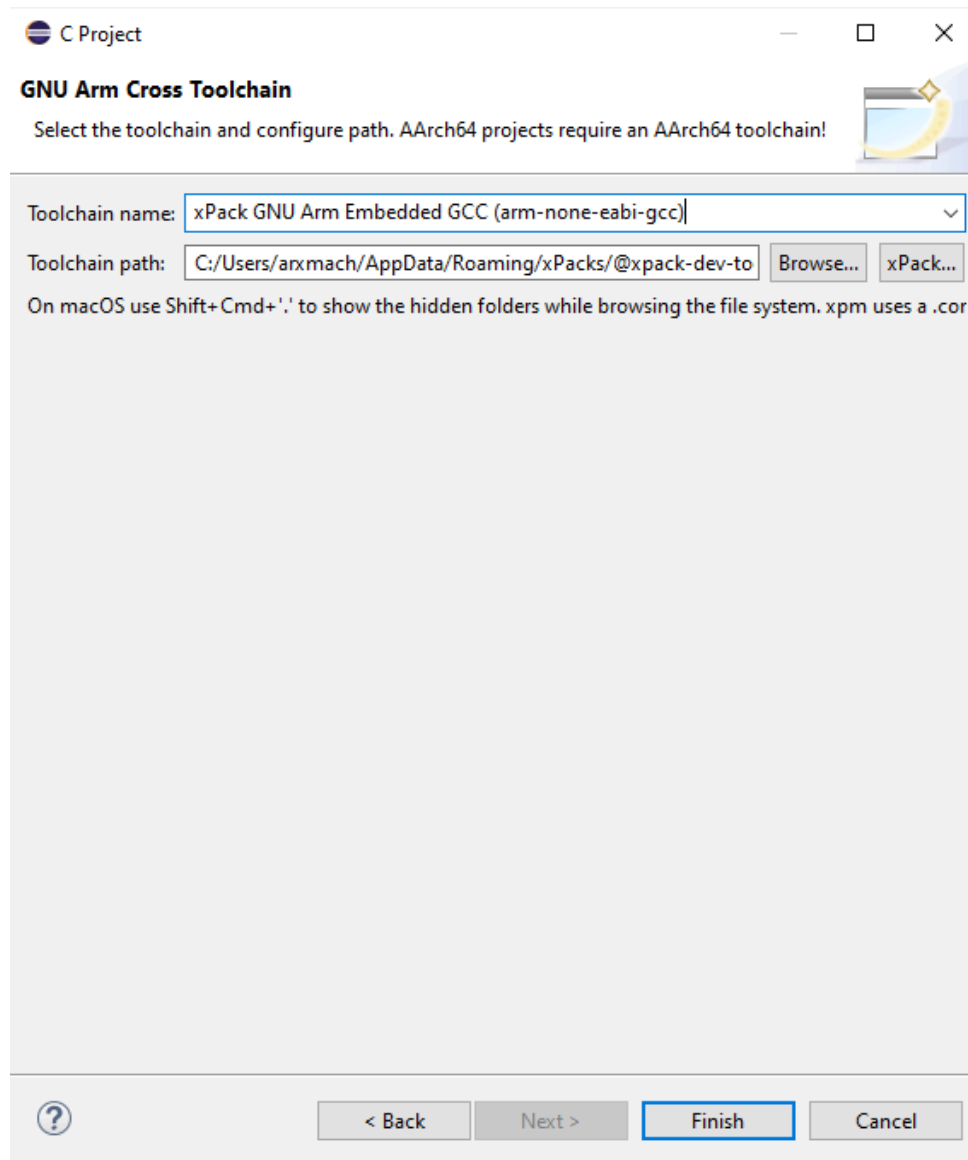


Fig 5: Setup the tool chain

(3) Toolchain name(GCC cross compiler)

(4) Toolchain path(GNU ARM Cross compiler path).

(Example:

C:/Users/arxmach/AppData/Roaming/xPacks/@xpack-dev-tools/arm-none-eabi-gcc/11.2.1-1.2.2/.content/bin)

6.0 Configure New Project

Editor: Eclipse IDE

STEP-1

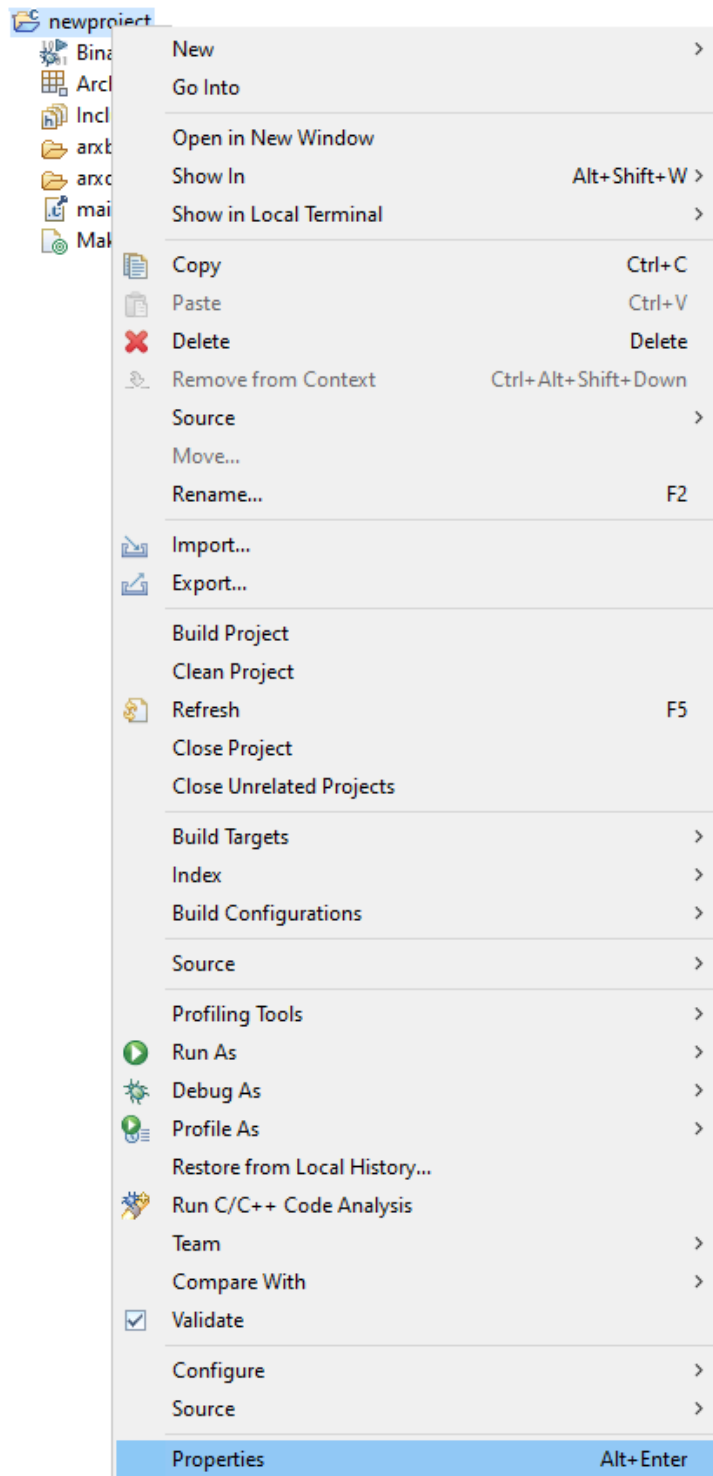


Fig 1: Right click on project → Properties

STEP-2

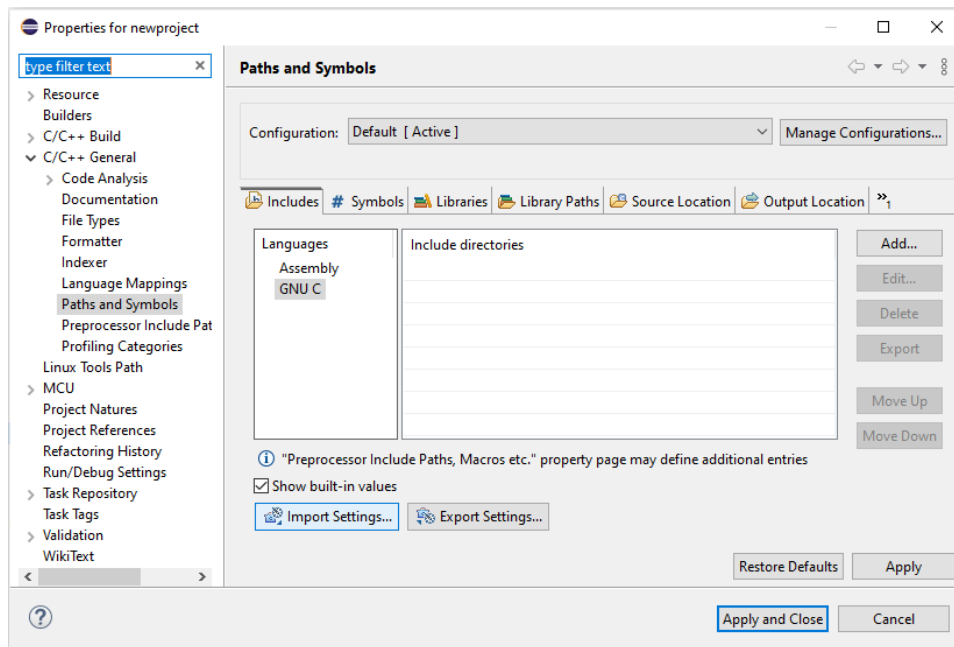


Fig 2: Import include paths settings from *.xml file

- I. Select "Paths and Symbols"
- II. Select Language
- III. Import Settings from arxos/tools/prjsetting.xml
- IV. Apply and Close

STEP-2.1

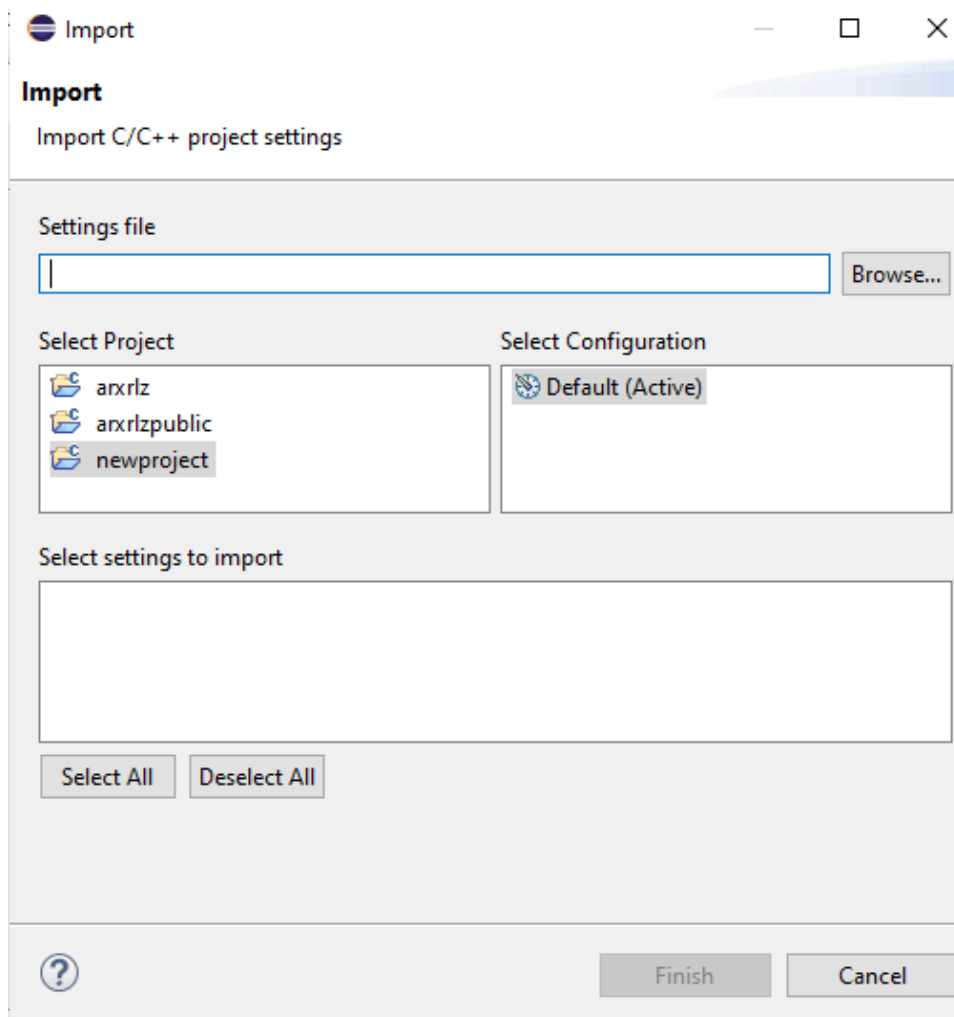


Fig 3: Import Include Paths settings from *.xml file

- I. Select the project
- II. Browse the project setting files(Example: arxos/tools/prjsetting.xml)
- III. Finish

STEP-2.2

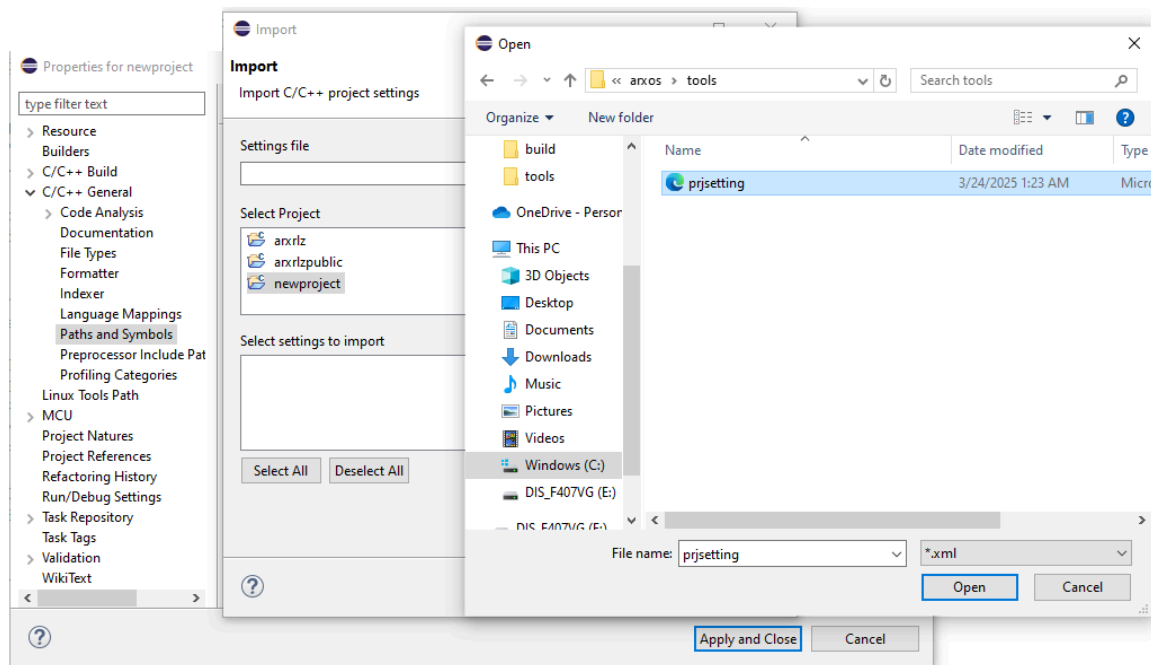


Fig 4: Select prjsetting.xml

7.0 Include Path Configuration(Manual)

Editor: Eclipse IDE

STEP-1

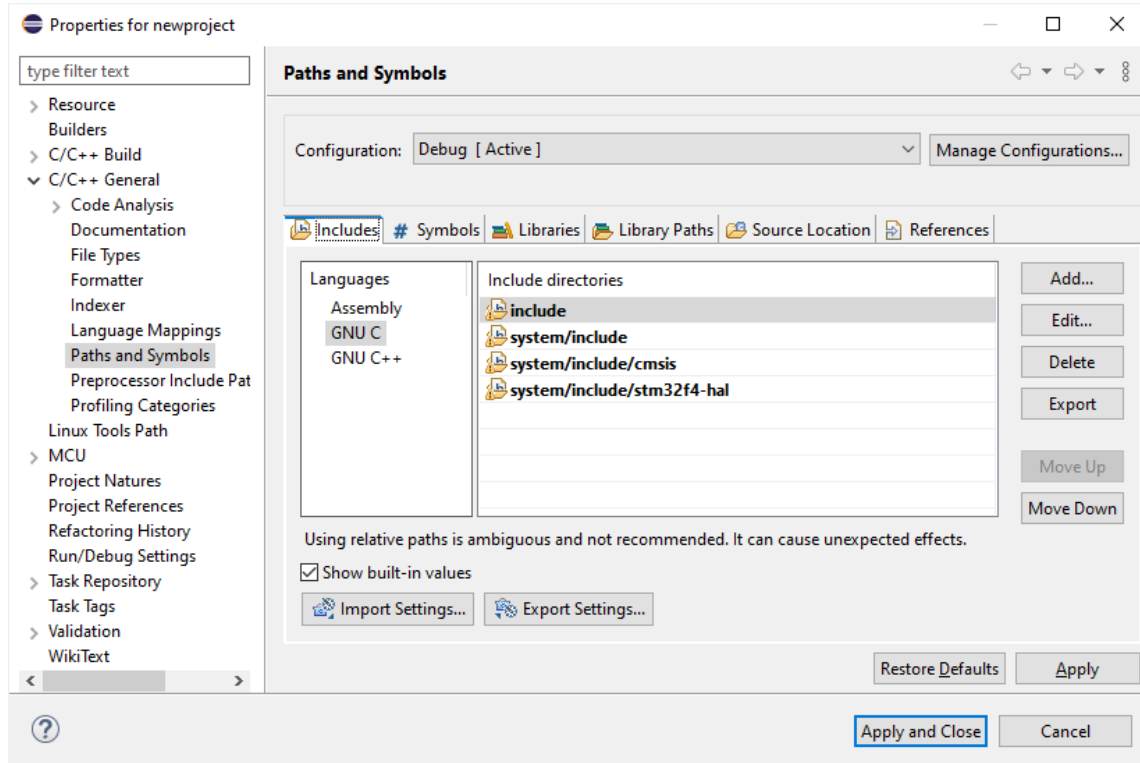


Fig 1: Navigate to your project properties.

Under C/C++ General → Paths and Symbols

- I. Select Include Tab
- II. Select Language
- III. Add the following entries.
 - arxos/arxif
 - arxos/services
 - arxos/config
 - arxos/usrsw/inclds
 - arxos/arch/arm/arm32/platforms/<SOC>/<Board>
 - arxos/arch/arm/arm32/platforms/<SOC>/<Board>drivers/uart
- IV. Apply and Close

8.0 Source Code Path Configuration

Editor: Eclipse IDE

STEP-1

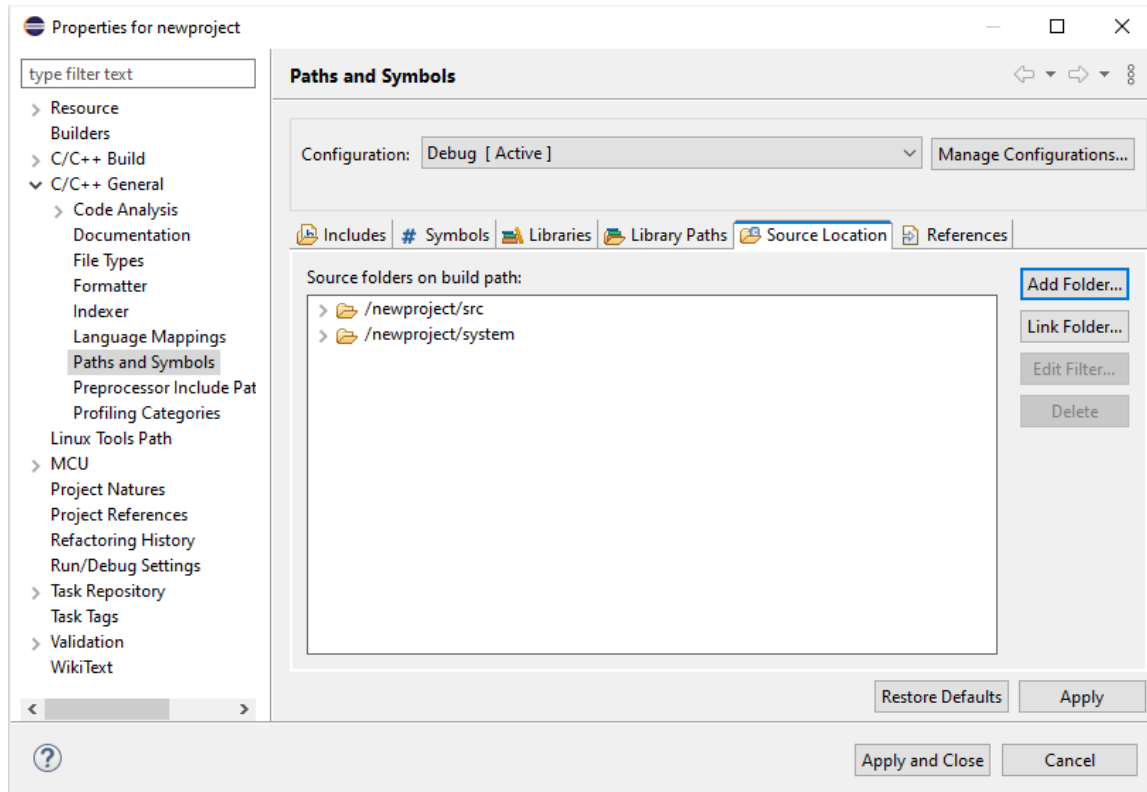


Fig 1: Navigate to your project properties.

- I. Under C/C++ General → Paths and Symbols
- II. Select Source Location Tab
- III. Select Add Folder

STEP-2

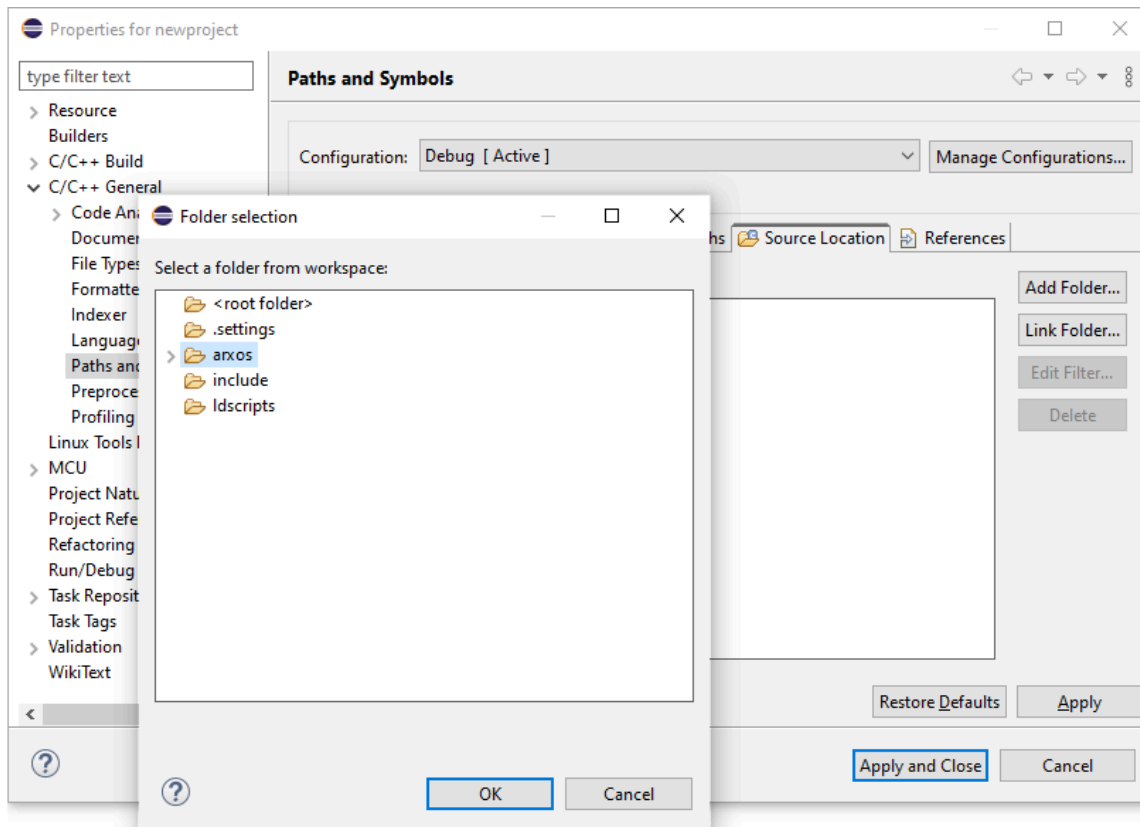


Fig 2: Select arxos source folder, Apply and Close.

9.0 Link ARX Static Library

Editor: Eclipse IDE

STEP-1

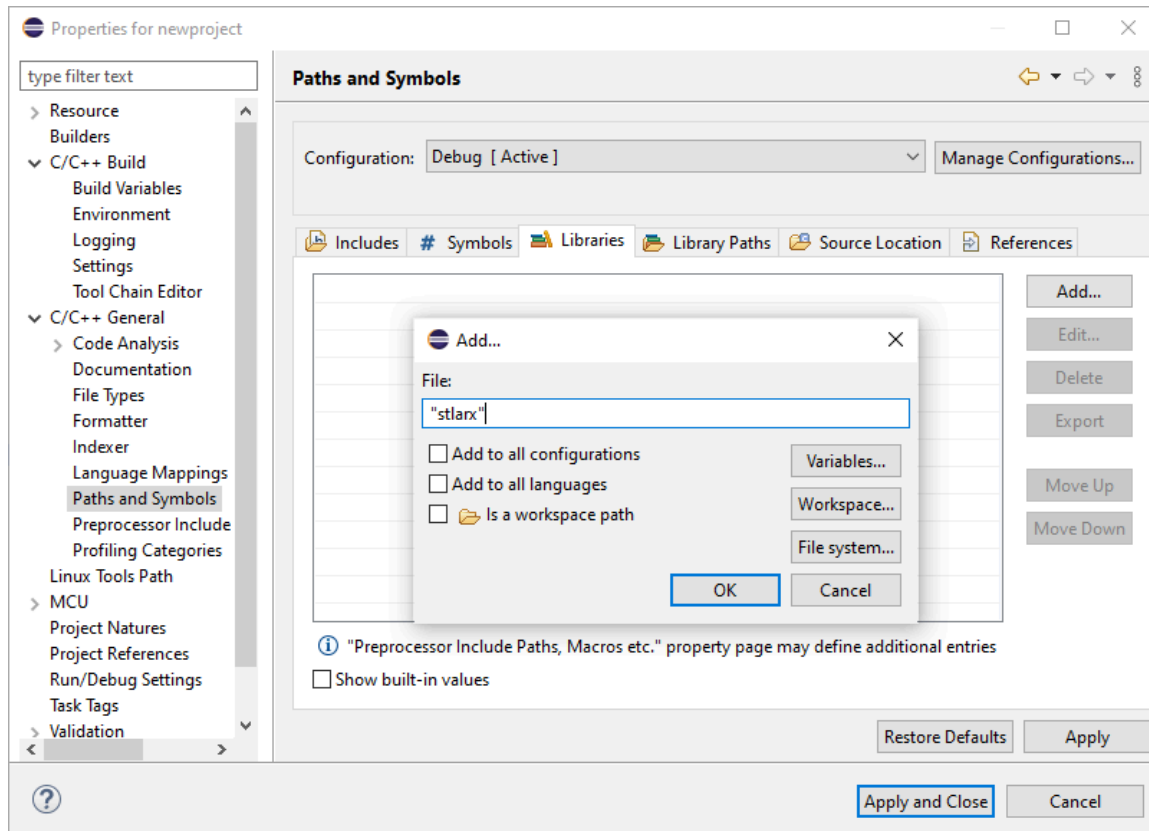


Fig 1: Navigate to your project properties.
Under C/C++ General → Paths and Symbols

- I. Select Libraries
- II. Select Add
- III. Type "stlrx"
- IV. OK.

STEP-2

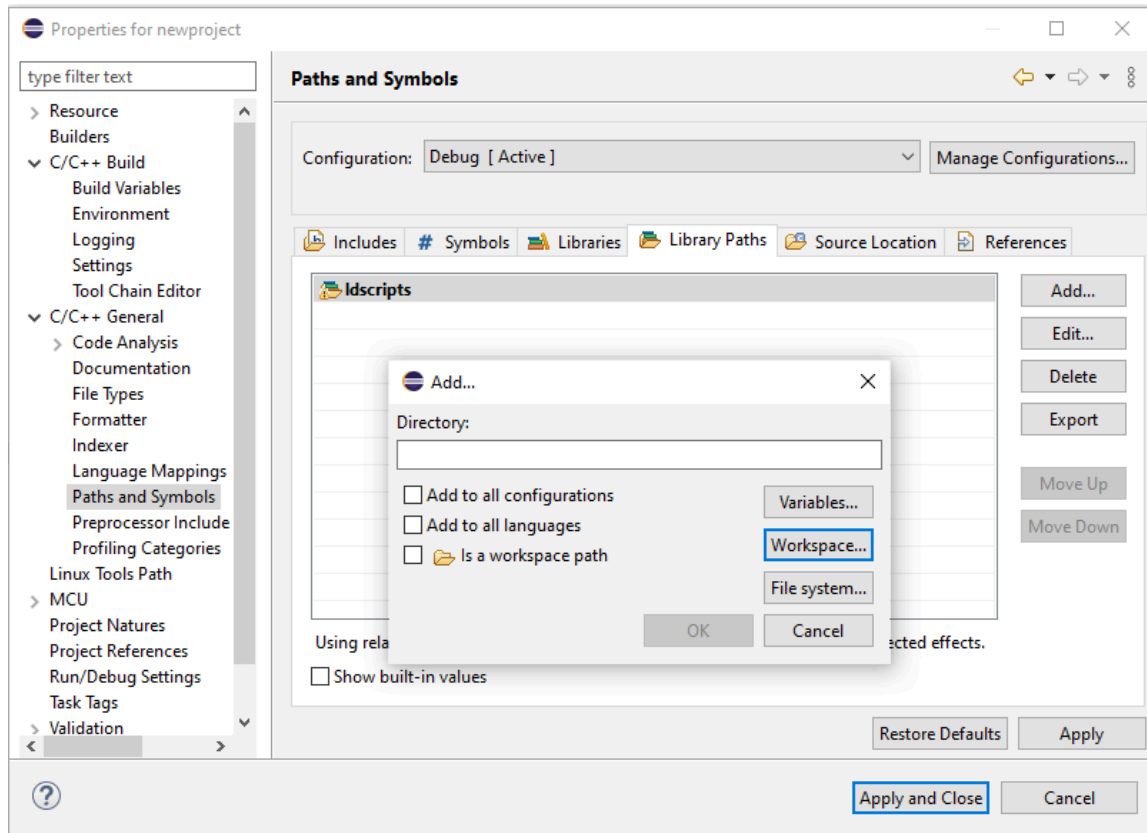


Fig 1: Select Libraries Path

- (1) Add
- (2) Browse this file from project workspace(Example: arxos/arch/libs/libstlarx.a)
- (3) Apply and Close

10.0 Target Settings

Editor: Eclipse IDE

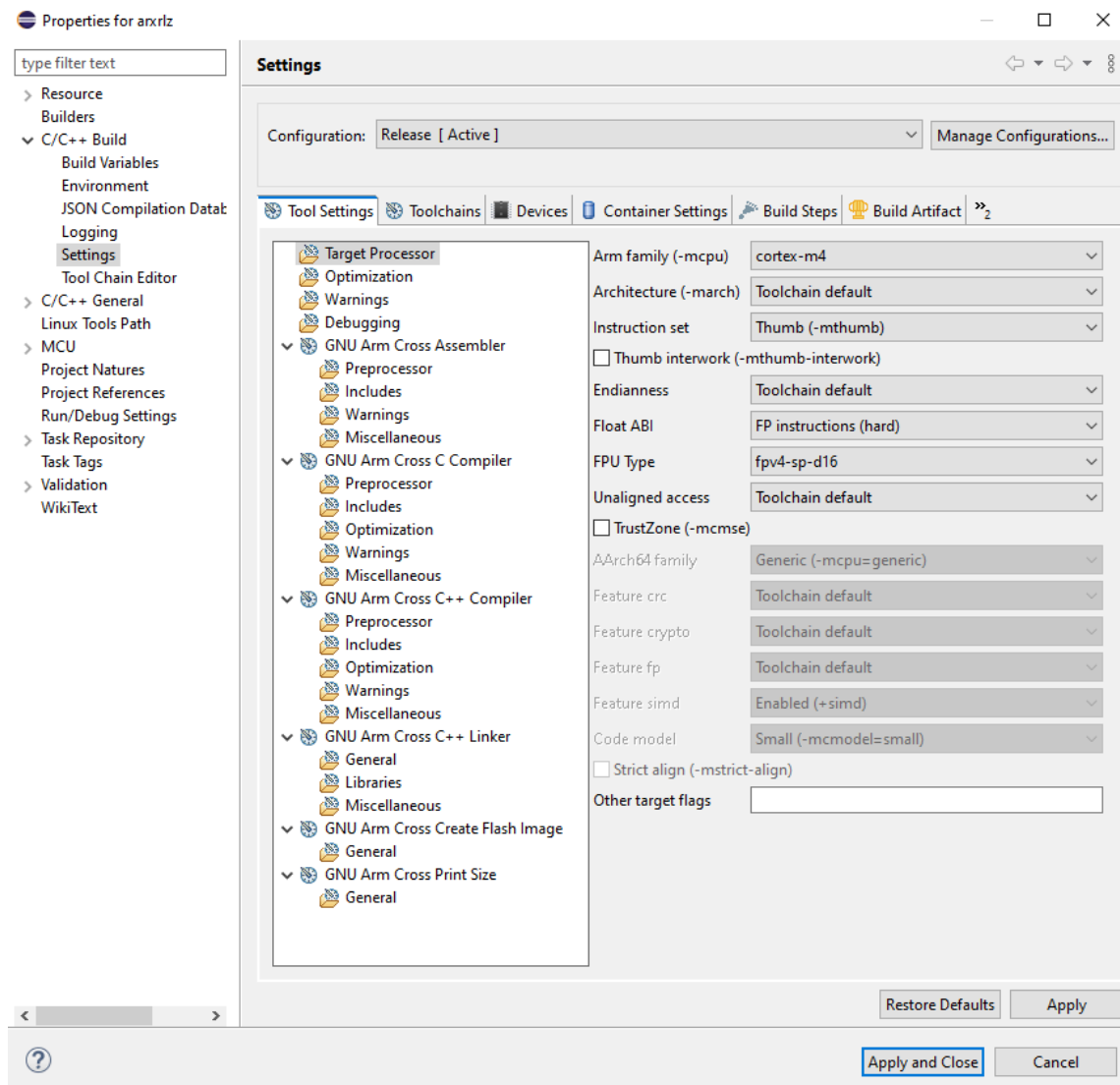


Fig 3: Select Architecture, FPU etc...

11.0 Build the ARX RTOS Project

After completing the project configuration, build the ARX RTOS project using Eclipse or the command line.

11.1 Build Using Eclipse IDE

In Eclipse:

1. Right-click the ARX RTOS project.
2. Select Build Project.
3. Monitor the Console view for build messages.
4. Confirm that the build completes without errors.

The existing ARX guide specifies Right-click on project → Select Build Project for the Eclipse build procedure.

11.2 Build Using Command Line

For Linux, the existing guide provides the following command-line build procedure.

Open a terminal and navigate to the ARX RTOS source directory:

```
ARXMACH@stl:~$ cd arxos  
ARXMACH@stl:~$ make
```

12.0 Flashing the Binary

STEP-1 Installing the Binary Download Tool

```
ARXMACH@stl:~$ sudo apt-get install stlink-tools
```

STEP-2 Writing the .bin File to Flash Memory

```
ARXMACH@stl: ~$ st-flash write /media/stlblr/F7B5-01AD/projects/stlertosfw.bin
0x8000000
st-flash 1.6.0
2022-02-04T19:54:55 INFO common.c: Loading device parameters....
2022-02-04T19:54:55 INFO common.c: Device connected is: F4 device, id 0x100f6413
2022-02-04T19:54:55 INFO common.c: SRAM size: 0x30000 bytes (192 KiB), Flash:
0x100000 bytes (1024 KiB) in pages of 16384 bytes
2022-02-04T19:54:55 INFO common.c: Attempting to write 41812 (0xa354) bytes to
stm32 address: 134217728 (0x8000000) Flash page at addr: 0x08008000
erasedEraseFlash - Sector:0x2 Size:0x4000 2022-02-04T19:54:56 INFO common.c:
Finished erasing 3 pages of 16384 (0x4000) bytes 2022-02-04T19:54:56 INFO
common.c: Starting Flash write for F2/F4/L4
2022-02-04T19:54:56 INFO flash_loader.c: Successfully loaded flash loader in sram
enabling 32-bit flash writes size: 32768 size: 9044
2022-02-04T19:54:56 INFO common.c: Starting verification of write complete
2022-02-04T19:54:57 INFO common.c: Flash written and verified! jolly good!
```

13.0 References

1. [arxos.pdf](#) — *Getting Started with ARXOS*
2. [arx_package_structure.pdf](#) — *Explains repository layout, directories, and files*
3. [arx_build_config.pdf](#) — *Building and Configuring ARXOS*
4. [arx_api_manual.pdf](#) — *ARXOS API Manual*
5. [arx_hal_guide.pdf](#) — *Hardware Abstraction Layer (HAL) Guide*
6. [arx_porting_guide.pdf](#) — *Porting ARXOS*
7. [arx_debugging_guide.pdf](#) — *Debugging ARXOS*
8. [arx_coding_standards.pdf](#) — *ARXOS Coding Standards*

14.0 Revision History

Version	Date	Author	Description
v1.0	30/10/2025	ARX Team	Initial draft

About ARX

ARX is a real-time operating system developed and licensed by ARX Software System.

✉ Contact: contact@arxos.in

Copyright and Trademark

© 2025 ARX Software System. All rights reserved.

This document and the software product described herein are provided under license.

No part of this document may be reproduced, extracted, or modified in any form without prior written permission from ARX Software System.

The software and its features may be used or copied only in accordance with the applicable license terms and conditions.

ARX and all related product names are trademarks or registered trademarks of ARX Software System. Other names or product references mentioned in this document may be the property of their respective owners.

Disclaimer

ARX Software System has made every effort to ensure the accuracy of the information contained in this document. However:

The information is provided “as is” without any warranty, express or implied.

This includes, but is not limited to, warranties of merchantability, fitness for a particular purpose, or non-infringement.

No guarantee is made that the software or documentation will be error-free, uninterrupted, or meet all specific requirements.

This software is provided under a commercial license agreement.

A free evaluation version may be offered for trial purposes only.

Evaluation software is intended solely for non-commercial use and must not be used in production or in revenue-generating environments without a valid license.

The user is solely responsible for:

- Conducting necessary evaluation, validation, and verification.
- Ensuring technical, functional, and performance requirements are fully met before deploying ARX in any operational, commercial, or production environment.

ARX Software System reserves the right to modify, update, or withdraw any content in this guide or software features at any time without prior notice.

Limitation of Liability

Under no circumstances shall the authors, copyright holders, or ARX Software System. be liable for any claim, damages, or liability arising from the use of, or inability to use, this software or documentation.

By using ARX software, you acknowledge and agree to these terms, and accept full responsibility for ensuring compliance with all applicable laws, regulations, and licensing conditions.

This document is about build environment setup and commands used here may vary, please ensure they are from secure source and safe prior installation